一、Linux 内核数据结构整体总结

1. 进程相关数据结构
    1) struct task_struct
    2) struct cred
    3) struct pid_link
    4) struct pid
    5) struct signal_struct
    6) struct rlimit
2. 内核中的队列/链表对象
    1) singly-linked lists
    2) singly-linked tail queues
    3) doubly-linked lists
    4) doubly-linked tail queues
3. 内核模块相关数据结构
    1) struct module
4. 文件系统相关数据结构
    1) struct file
    2) struct inode
    3) struct stat
    4) struct fs_struct
    5) struct files_struct
    6) struct fdtable

7) struct dentry

8) struct vfsmount

9) struct nameidata

10) struct super_block

11) struct file_system_type

5. 内核安全相关数据结构

1) struct security_operations

2) struct kprobe

3) struct jprobe

4) struct kretprobe

5) struct kretprobe_instance

6) struct kretprobe_blackpoint 、struct kprobe_blacklist_entry

7) struct linux_binprm

8) struct linux_binfmt

6. 系统网络状态相关的数据结构

1) struct ifconf

2) struct ifreq

3) struct socket

4) struct sock

5) struct proto_ops

6) struct inet_sock

7) struct sockaddr

7. 系统内存相关的数据结构

1) struct mm_struct

2) struct vm_area_struct

3) struct pg_data_t

4) struct zone

5) struct page

8. 中断相关的数据结构

1) struct irq_desc

2) struct irq_chip

3) struct irqaction

9. 进程间通信(IPC)相关数据结构

1) struct ipc_namespace

2) struct ipc_ids

3) struct kern_ipc_perm

4) struct sysv_sem

5) struct sem_queue

6) struct msg_queue

7) struct msg_msg

8) struct msg_sender

9) struct msg_receiver

10) struct msqid_ds

10. 命名空间(namespace)相关数据结构

1) struct pid_namespace

2) struct pid、struct upid

3) struct nsproxy

4) struct mnt_namespace

二、Linux 内核大厂面试题精选

1) Linux 中主要有哪几种内核锁?

2) Linux 中的用户模式和内核模式是什么含意?

3) 怎样申请大块内核内存?

4) 用户进程间通信主要哪几种方式?

5) 通过伙伴系统申请内核内存的函数有哪些?

6) 通过 slab 分配器申请内核内存的函数有?

7) Linux 的内核空间和用户空间是如何划分的(以 32 位系统为例)?

8) vmalloc()申请的内存有什么特点?

9) 用户程序使用 malloc()申请到的内存空间在什么范围?

10) 在支持并使能 MMU 的系统中，Linux 内核和用户程序分别运行在物理地址模式还是虚拟地址模式?

11) ARM 处理器是通过几级也表进行存储空间映射的?

12) Linux 是通过什么组件来实现支持多种文件系通的?

13) Linux 虚拟文件系统的关键数据结构有哪些? (至少写出四个)

14）对文件或设备的操作函数保存在那个数据结构中？

15）Linux 中的文件包括哪些？

16）创建进程的系统调用有那些？

17）调用 schedule()进行进程切换的方式有几种？

18）Linux 调度程序是根据进程的动态优先级还是静态优先级来调度进程的？

19）进程调度的核心数据结构是哪个？

20）如何加载、卸载一个模块？

21）模块和应用程序分别运行在什么空间？

22）Linux 中的浮点运算由应用程序实现还是内核实现？

23）模块程序能否使用可链接的库函数？

24）TLB 中缓存的是什么内容？

25）Linux 中有哪几种设备？

26）字符设备驱动程序的关键数据结构是哪个？

27）设备驱动程序包括哪些功能函数？

28）如何唯一标识一个设备？

29）Linux 通过什么方式实现系统调用？

30）Linux 软中断和工作队列的作用是什么？

1、Linux 中主要有哪几种内核锁？

Linux 的同步机制从 2.0 到 2.6 以来不断发展完善。从最初的原子操作，到后来的信号量，从大内核锁到今天的自旋锁。这些同步机制的发展伴随 Linux 从单处理器到对称多处理器的过渡；

伴随着从非抢占内核到抢占内核的过度。Linux 的锁机制越来越有效，也越来越复杂。

Linux 的内核锁主要是自旋锁和信号量。

自旋锁最多只能被一个可执行线程持有，如果一个执行线程试图请求一个已被争用（已经被持有）的自旋锁，那么这个线程就会一直进行忙循环——旋转——等待锁重新可用。要是锁未被争用，请求它的执行线程便能立刻得到它并且继续进行。自旋锁可以在任何时刻防止多于一个的执行线程同时进入临界区。

Linux 中的信号量是一种睡眠锁。如果有一个任务试图获得一个已被持有的信号量时，信号量会将其推入等待队列，然后让其睡眠。这时处理器获得自由去执行其它代码。当持有信号量的进程将信号量释放后，在等待队列中的一个任务将被唤醒，从而便可以获得这个信号量。信号量的睡眠特性，使得信号量适用于锁会被长时间持有的情况；只能在进程上下文中使用，因为中断上下文中是不能被调度的；另外当代码持有信号量时，不可以再持有自旋锁。

Linux 内核中的同步机制：原子操作、信号量、读写信号量和自旋锁的 API，另外一些同步机制，包括大内核锁、读写锁、大读者锁、RCU (Read-Copy Update，顾名思义就是读-拷贝修改)，和顺序锁。

2、Linux 中的用户模式和内核模式是什么含意？

MS-DOS 等操作系统在单一的 CPU 模式下运行，但是一些类 Unix 的操作系统则使用了双模式，可以有效地实现时间共享。在 Linux 机器上，CPU 要么处于受信任的内核模式，要么处于受限制的用户模式。除了内核本身处于内核模式以外，所有的用户进程都运行在用户模式之中。

内核模式的代码可以无限制地访问所有处理器指令集以及全部内存和 I/O 空间。如果用户模式的进程要享有此特权，它必须通过系统调用向设备驱动程序或其他内核模式的代码发出请求。另外，用户模式的代码允许发生缺页，而内核模式的代码则不允许。

在 2.4 和更早的内核中，仅仅用户模式的进程可以被上下文切换出局，由其他进程抢占。除非发生以下两种情况，否则内核模式代码可以一直独占 CPU：

(1) 它自愿放弃 CPU；

(2) 发生中断或异常。

2.6 内核引入了内核抢占，大多数内核模式的代码也可以被抢占。

3、怎样申请大块内核内存？

在 Linux 内核环境下，申请大块内存的成功率随着系统运行时间的增加而减少，虽然可以通过 vmalloc 系列调用申请物理不连续但虚拟地

址连续的内存，但毕竟其使用效率不高且在 32 位系统上 vmalloc 的内存地址空间有限。所以，一般的建议是在系统启动阶段申请大块内存，但是其成功的概率也只是比较高而已，而不是 100%。如果程序真的比较在意这个申请的成功与否，只能退用“启动内存”（Boot Memory）。下面就是申请并导出启动内存的一段示例代码：

```
void*  x_bootmem  =  NULL;
EXPORT_SYMBOL(x_bootmem);

unsigned  long  x_bootmem_size  =  0;
EXPORT_SYMBOL(x_bootmem_size);

static  int  __init x_bootmem_setup(char  *str)
{
                x_bootmem_size  =  memparse(str,  &str);
                x_bootmem  =  alloc_bootmem(x_bootmem_size);
                printk("Reserved %lu bytes from %p for
x\n",  x_bootmem_size,  x_bootmem);

                return  1;
}
__setup("x-bootmem=",  x_bootmem_setup);
```

可见其应用还是比较简单的，不过利弊总是共生的，它不可避免也有其自身的限制：

内存申请代码只能连接进内核，不能在模块中使用。

被申请的内存不会被页分配器和 slab 分配器所使用和统计，也就是说它处于系统的可见内存之外，即使在将来的某个地方你释放了它。

一般用户只会申请一大块内存，如果需要在其上实现复杂的内存管理则需要自己实现。

在不允许内存分配失败的场合，通过启动内存预留内存空间将是我们唯一的选择。

4、用户进程间通信主要哪几种方式？

（1）管道（Pipe）：管道可用于具有亲缘关系进程间的通信，允许一个进程和另一个与它有共同祖先的进程之间进行通信。

（2）命名管道（named pipe）：命名管道克服了管道没有名字的限制，因此，除具有管道所具有的功能外，它还允许无亲缘关系进程间的通信。命名管道在文件系统中有对应的文件名。命名管道通过命令 mkfifo 或系统调用 mkfifo 来创建。

（3）信号（Signal）：信号是比较复杂的通信方式，用于通知接受进程有某种事件发生，除了用于进程间通信外，进程还可以发送

信号给进程本身；linux 除了支持 Unix 早期信号语义函数 sigal 外，还支持语义符合 Posix.1 标准的信号函数 sigaction（实际上，该函数是基于 BSD 的，BSD 为了实现可靠信号机制，又能够统一对外接口，用 sigaction 函数重新实现了 signal 函数）。

（4）消息（Message）队列：消息队列是消息的链接表，包括 Posix 消息队列 system V 消息队列。有足够权限的进程可以向队列中添加消息，被赋予读权限的进程则可以读走队列中的消息。消息队列克服了信号承载信息量少，管道只能承载无格式字节流以及缓冲区大小受限等缺

（5）共享内存：使得多个进程可以访问同一块内存空间，是最快的可用 IPC 形式。是针对其他通信机制运行效率较低而设计的。往往与其它通信机制，如信号量结合使用，来达到进程间的同步及互斥。

（6）信号量（semaphore）：主要作为进程间以及同一进程不同线程之间的同步手段。

（7）套接字（Socket）：更为一般的进程间通信机制，可用于不同机器之间的进程间通信。起初是由 Unix 系统的 BSD 分支开发出来的，但现在一般可以移植到其它类 Unix 系统上：Linux 和 System V 的变种都支持套接字。

5、通过伙伴系统申请内核内存的函数有哪些？

在物理页面管理上实现了基于区的伙伴系统（zone based buddy system）。对不同区的内存使用单独的伙伴系统(buddy system)管理，而且独立地监控空闲页。相应接口 alloc_pages(gfp_mask, order)，_ _get_free_pages(gfp_mask, order)等。

补充知识：

1.原理说明

Linux 内核中采　用了一种同时适用于 32 位和 64 位系统的内　存分页模型，对于 32 位系统来说，两级页表足够用了，而在 x86_64 系　统中，用到了四级页表。

* 页全局目录(Page Global Directory)
* 页上级目录(Page Upper Directory)
* 页中间目录(Page Middle Directory)
* 页表(Page Table)

页全局目录包含若干页上级目录的地址，页上级目录又依次包含若干页中间目录的地址，而页中间目录又包含若干页表的地址，每一个页表项指　向一个页框。Linux 中采用 4KB 大小的　页框作为标准的内存分配单元。

多级分页目录结构

1.1.伙伴系统算法

在实际应用中，经常需要分配一组连续的页框，而频繁地申请和释放不同大小的连续页框，必然导致在已分配页框的内存块中分散了许多小块的　空闲页框。这样，即使这些页框是空闲的，其他需要分配连续页框的应用也很难得到满足。

为了避免出现这种情况，Linux 内核中引入了伙伴系统算法(buddy system)。把所有的空闲页框分组为 11 个　块链表，每个块链表分别包含大小为 1，2，4，8，16，32，64，128，256，512 和 1024 个连续页框的页框块。最大可以申请 1024 个连　续页框，对应 4MB 大小的连续内存。每个页框块的第一个页框的物理地址是该块大小的整数倍。

假设要申请一个 256 个页框的块，先从 256 个页框的链表中查找空闲块，如果没有，就去 512 个　页框的链表中找，找到了则将页框块分为 2 个 256 个　页框的块，一个分配给应用，另外一个移到 256 个页框的链表中。如果 512 个页框的链表中仍没有空闲块，继续向 1024 个页　框的链表查找，如果仍然没有，则返回错误。

页框块在释放时，会主动将两个连续的页框块合并为一个较大的页框块。

1.2.slab 分配器

slab 分配器源于　Solaris 2.4　的　分配算法，工作于物理内存页框分配器之上，管理特定大小对象的缓存，进行快速而高效的内存分配。

slab 分配器为每种使用的内核对象建立单独的缓冲区。Linux 内核已经采用了伙伴系统管理物理内存页框，因此 slab 分配器直接工作于伙伴系 统之上。每种缓冲区由多个 slab 组成，每个 slab 就是一组连续的物理内存页框，被划分成了固定数目的对象。根据对象大小的不同，缺省情况下一个 slab 最多可以由 1024 个页框构成。出于对齐 等其它方面的要求，slab 中分配给对象的内存可能大于用户要求的对象实际大小，这会造成一定的 内存浪费。

2.常用内存分配函数

2.1.__get_free_pages

unsigned long __get_free_pages(gfp_t gfp_mask, unsigned int order)

__get_free_pages 函数是最原始的内存分配方式，直接从伙伴系统中获取原始页框，返回值为第一个页框的起始地址。__get_free_pages 在实现上只是封装了 alloc_pages 函 数，从代码分析，alloc_pages 函数会分配长度为 1<

2.2.kmem_cache_alloc

struct kmem_cache *kmem_cache_create(const char *name, size_t size,

size_t align, unsigned long flags,

void (*ctor)(void*, struct kmem_cache *, unsigned long),

void (*dtor)(void*, struct kmem_cache *, unsigned long))

void *kmem_cache_alloc(struct kmem_cache *c, gfp_t flags)

kmem_cache_create/ kmem_cache_alloc 是基于 slab 分配器的一种内存分配方式，适用于反复分配释放同一大小内存块的场合。首先用 kmem_cache_create 创建一个高速缓存区域，然后用 kmem_cache_alloc 从 该高速缓存区域中获取新的内存块。kmem_cache_alloc 一次能分配的最大内存由 mm/slab.c 文件中的 MAX_OBJ_ORDER 宏定义，在默认的 2.6.18 内核版本中，该宏定义为 5， 于是一次最多能申请 1<<5 * 4KB 也就是 128KB 的 连续物理内存。分析内核源码发现，kmem_cache_create 函数的 size 参数大于 128KB 时会调用 BUG()。测试结果验证了分析结果，用 kmem_cache_create 分 配超过 128KB 的内存时使内核崩溃。

2.3.kmalloc

void *kmalloc(size_t size, gfp_t flags)

kmalloc 是内核中最常用的一种内存分配方式，它通过调用 kmem_cache_alloc 函数来实现。kmalloc 一次最多能申请的内存大小由 include/linux/Kmalloc_size.h 的 内容来决定，在默认的 2.6.18 内核版本中，kmalloc 一 次最多能申请大小为 131702B 也就是 128KB 字 节的连续物理内存。测试结果表明，如果试图用 kmalloc 函数分配大于 128KB 的内存，编译不能通过。

2.4.vmalloc

void *vmalloc(unsigned long size)

前面几种内存分配方式都是物理连续的，能保证较低的平均访问时间。但是在某些场合中，对内存区的请求不是很频繁，较高的内存访问时间也 可以接受，这是就可以分配一段线性连续，物理不连续的地址，带来的好处是一次可以分配较大块的内存。图 3-1 表 示的是 vmalloc 分配的内存使用的地址范围。vmalloc 对 一次能分配的内存大小没有明确限制。出于性能考虑，应谨慎使用 vmalloc 函数。在测试过程中， 最大能一次分配 1GB 的空间。

Linux 内核部分内存分布

2.5.dma_alloc_coherent

void *dma_alloc_coherent(struct device *dev, size_t size,

ma_addr_t *dma_handle, gfp_t gfp)

DMA 是一种硬件机制，允许外围设备和主存之间直接传输 IO 数据，而不需要 CPU 的参与，使用 DMA 机制能大幅提高与设备通信的 吞吐量。DMA 操作中，涉及到 CPU 高速缓 存和对应的内存数据一致性的问题，必须保证两者的数据一致，在 x86_64 体系结构中，硬件已经很 好的解决了这个问题，dma_alloc_coherent 和 __get_free_pages 函数实现差别不大，前者实际是调用 __alloc_pages 函 数来分配内存，因此一次分配内存的大小限制和

后者一样。__get_free_pages 分配的内　存同样可以用于 DMA 操作。测试结果证明，dma_alloc_coherent 函　数一次能分配的最大内存也为 4M。

2.6. ioremap

void * ioremap (unsigned long offset, unsigned long size)

ioremap 是一种更直接的内存“分配”方式，使用时直接指定物理起始地址和需要分配内存的大小，然后将该段　物理地址映射到内核地址空间。ioremap 用到的物理地址空间都是事先确定的，和上面的几种内存　分配方式并不太一样，并不是分配一段新的物理内存。ioremap 多用于设备驱动，可以让 CPU 直接访问外部设备的 IO 空间。ioremap 能映射的内存由原有的物理内存空间决定，所以没有进行测试。

2.7. Boot Memory

如果要分配大量的连续物理内存，上述的分配函数都不能满足，就只能用比较特殊的方式，在 Linux 内　核引导阶段来预留部分内存。

2.7.1. 在内核引导时分配内存

void* alloc_bootmem(unsigned long size)

可以在 Linux 内核引导过程中绕过伙伴系统来分配大块内存。使用方法是在 Linux 内核引导时，调用 mem_init 函数之前　用 alloc_bootmem 函数申请指定大小的内存。如果需要在其他地方调用这块内存，可以将 alloc_bootmem 返回的内存首地址通过 EXPORT_SYMBOL 导　出，然后就可以使用这块内存了。这种内存分配方式的缺点是，申请内存的代码必须在链接到内核中的代码里才能使用，因此必须重新编译内核，而且内存管理系统　看不到这部分内存，需要用户自行管理。测试结果表明，重新编译内核后重启，能够访问引导时分配的内存块。

2.7.2.通过内核引导参数预留顶部内存

在 Linux 内核引导时，传入参数“mem=size”保留顶部的内存区间。比如系统有 256MB 内　存，参数“mem=248M”会预留顶部的 8MB 内存，进入系统后可以调用 ioremap(0xF800000，0x800000)来申请这段内存。

3.几种分配函数的比较

分配原理最大内存其他

__get_free_pages 直接对页框进行操作 4MB 适用于分配较大量的连续物理内存

kmem_cache_alloc 基于 slab 机制实现 128KB 适合需要频繁申请释放相同大小内存块时使用

kmalloc 基于 kmem_cache_alloc 实现 128KB 最常见的分配方式，需要小于页框大小的内存时可以使用

vmalloc 建立非连续物理内存到虚拟地址的映射物理不连续，适合需要大内存，但是对地址连续性没有要求的场合

dma_alloc_coherent 基于__alloc_pages 实现 4MB 适用于 DMA 操　作

ioremap 实现已知物理地址到虚拟地址的映射适用于物理地址已知的场合，如设备驱动

alloc_bootmem 在启动 kernel 时，预留一段内存，内核看不见小于物理内存大小，内存管理要求较高

三、最新字节跳动 linux 后台开发

1. C++智能指针如何解决内存泄露问题.

1.shared_ptr 共享的智能指针

std::shared_ptr 使用引用计数，每一个 shared_ptr 的拷贝都指向相同的内存。在最后一个 shared_ptr 析构的时候，内存才会被释放。

可以通过构造函数、std_make_shared 辅助函数和 reset 方法来初始化 shared_ptr：

// 构造函数初始化

std::shared_ptrp (new int(1)) ;

std::shared_ptrp2 = p ;

// 对于一个未初始化的智能指针，可以通过 reset 方法来初始化。

std::shared_ptrptr; ptr.reset (new int (1)) ;

if (ptr) {cout << “ptr is not null.\n” ; }

不能将一个原始指针直接赋值给一个智能指针：

std::shared_ptrp ＝ new int(1) ;// 编译报错，不允许直接赋值

获取原始指针：

通过 get 方法来返回原始指针

```
std::shared_ptrptr ( new int(1) ) ;

int * p =ptr.get () ;
```

指针删除器：

智能指针初始化可以指定删除器

```
void DeleteIntPtr ( int * p ) {

delete p ;

}

std::shared_ptrp ( new int , DeleteIntPtr ) ;
```

当 p 的引用技术为 0 时，自动调用删除器来释放对象的内存。删除器也可以是一个 lambda 表达式，例如

```
std::shared_ptrp ( new int , [](int * p){delete p} ) ;
```

注意事项：

(1).不要用一个原始指针初始化多个 shared_ptr。

(2).不要再函数实参中创建 shared_ptr，在调用函数之前先定义以及初始化它。

(3).不要将 this 指针作为 shared_ptr 返回出来。

(4).要避免循环引用。

2.unique_ptr 独占的智能指针

unique_ptr 是一个独占的智能指针，他不允许其他的智能指针共享其内部的指针，不允许通过赋值将一个 unique_ptr 赋值给另外一个 unique_ptr。

unique_ptr 不允许复制，但可以通过函数返回给其他的 unique_ptr，还可以通过 std::move 来转移到其他的 unique_ptr，这样它本身就不再拥有原来指针的所有权了。

如果希望只有一个智能指针管理资源或管理数组就用 unique_ptr，如果希望多个智能指针管理同一个资源就用 shared_ptr。

3.weak_ptr 弱引用的智能指针

弱引用的智能指针 weak_ptr 是用来监视 shared_ptr 的，不会使引用计数加一，它不管理 shared_ptr 内部的指针，主要是为了监视 shared_ptr 的生命周期，更像是 shared_ptr 的一个助手。

weak_ptr 没有重载运算符*和->，因为它不共享指针，不能操作资源，主要是为了通过 shared_ptr 获得资源的监测权，它的构造不会增加引用计数，它的析构不会减少引用计数，纯粹只是作为一个旁观者来监视 shared_ptr 中关离得资源是否存在。

weak_ptr 还可以用来返回 this 指针和解决循环引用的问题。

2. linux 中软连接和硬链接的区别.

原理上，硬链接和源文件的 inode 节点号相同，两者互为硬链接。软连接和源文件的 inode 节点号不同，进而指向的 block 也不同，软连接 block 中存放了源文件的路径名。

实际上，硬链接和源文件是同一份文件，而软连接是独立的文件，类似于快捷方式，存储着源文件的位置信息便于指向。

使用限制上，不能对目录创建硬链接，不能对不同文件系统创建硬链接，不能对不存在的文件创建硬链接；可以对目录创建软连接，可以跨文件系统创建软连接，可以

对不存在的文件创建软连接。

3. TCP 的拥塞控制机制是什么？请简单说说

我们知道 TCP 通过一个定时器（timer）采样了 RTT 并计算 RTO，但是，如果网络上的延时突然增加，那么，TCP 对这个事做出的应对只有重传数据，然而重传会导致网络的负担更重，于是会导致更大的延迟以及更多的丢包，这就导致了恶性循环，最终形成“网络风暴”—— TCP 的拥塞控制机制就是用于应对这种情况。

首先需要了解一个概念，为了在发送端调节所要发送的数据量，定义了一个“拥塞窗口”（Congestion Window），在发送数据时，将拥塞窗口的大小与接收端 ack 的窗口大小做比较，取较小者作为发送数据量的上限。

拥塞控制主要是四个算法：

1.慢启动：意思是刚刚加入网络的连接，一点一点地提速，不要一上来就把路占满。

连接建好的开始先初始化 cwnd = 1，表明可以传一个 MSS 大小的数据。

每当收到一个 ACK，cwnd++; 呈线性上升

每当过了一个 RTT，cwnd = cwnd*2; 呈指数让升

阈值 ssthresh（slow start threshold），是一个上限，当 cwnd >= ssthresh 时，就会进入“拥塞避免算法”

2.拥塞避免：当拥塞窗口 cwnd 达到一个阈值时，窗口大小不再呈指数上升，而是以线性上升，避免增长过快导致网络拥塞。

每当收到一个 ACK，cwnd = cwnd + 1/cwnd

每当过了一个 RTT，cwnd = cwnd + 1

拥塞发生：当发生丢包进行数据包重传时，表示网络已经拥塞。分两种情况进行处理：

等到 RTO 超时，重传数据包

sshthresh = cwnd /2

cwnd 重置为 1

3.进入慢启动过程

在收到 3 个 duplicate ACK 时就开启重传，而不用等到 RTO 超时

sshthresh = cwnd = cwnd /2

进入快速恢复算法——Fast Recovery

4.快速恢复：至少收到了 3 个 Duplicated Acks，说明网络也不那么糟糕，可以快速恢复。

cwnd = sshthresh + 3 * MSS （3 的意思是确认有 3 个数据包被收到了）

重传 Duplicated ACKs 指定的数据包

如果再收到 duplicated Acks，那么 cwnd = cwnd +1

如果收到了新的 Ack，那么，cwnd = sshthresh ，然后就进入了拥塞避免的算法了。

4. 如何理解 IO 多路复用的三种机制 Select，Poll，Epoll?

1.Select

首先先分析一下 select 函数

```
int select(

int maxfdp1,

fd_set *readset,

fd_set *writeset,

fd_set *exceptset,
```

const struct timeval *timeout

);

【参数说明】

int maxfdp1 指定待测试的文件描述字个数，它的值是待测试的最大描述字加 1。

fd_set *readset , fd_set *writeset , fd_set *exceptset

fd_set 可以理解为一个集合，这个集合中存放的是文件描述符(file descriptor)，即 文件句柄。中间的三个参数指定我们要让内核测试读、写和异常条件的文件描述符集合。 如果对某一个的条件不感兴趣，就可以把它设为空指针。

const struct timeval *timeout timeout 告知内核等待所指定文件描述符集合中的任 何一个就绪可花多少时间。其 timeval 结构用于指定这段时间的秒数和微秒数。

【返回值】

int 若有就绪描述符返回其数目，若超时则为 0，若出错则为-1

select 运行机制

select()的机制中提供一种 fd_set 的数据结构，实际上是一个 long 类型的数组，每一 个数组元素都能与一打开的文件句柄（不管是 Socket 句柄,还是其他文件或命名管道或 设备句柄）建立联系，建立联系的工作由程序员完成，当调用 select()时，由内核根 据 IO 状态修改 fd_set 的内容，由此来通知执行了 select()的进程哪一 Socket 或文件 可读。

从流程上来看，使用 select 函数进行 IO 请求和同步阻塞模型没有太大的区别，甚至还 多了添加监视 socket，以及调用 select 函数的额外操作，效率更差。但是，使用 select 以后最大的优势是用户可以在一个线程内同时处理多个 socket 的 IO 请求。用户可以注 册多个 socket，然后不断地调用 select 读取被激活的 socket，即可达到在同一个线程 内同时处理多个 IO 请求的目的。而在同步阻塞模型中，必须通过多线程的方式才能达 到这个目的。

select 机制的问题

每次调用 select,都需要把 fd_set 集合从用户态拷贝到内核态，如果 fd_set 集合很 大时，那这个开销也很大

同时每次调用 select 都需要在内核遍历传递进来的所有 fd_set，如果 fd_set 集合很 大时，那这个开销也很大

为了减少数据拷贝带来的性能损坏，内核对被监控的 fd_set 集合大小做了限制，并且 这个是通过宏控制的，大小不可改变(限制为 1024).

2.Poll

poll 的机制与 select 类似，与 select 在本质上没有多大差别，管理多个描述符也是 进行轮询，根据描述符的状态进行处理，但是 poll 没有最大文件描述符数量的限制。 也就是说，poll 只解决了上面的问题 3，并没有解决问题 1，2 的性能开销问题。

下面是 pll 的函数原型：

```
int poll(struct pollfd *fds, nfds_t nfds, int timeout);

typedef struct pollfd {
```

int fd; // 需要被检测或选择的文件描述符

short events; // 对文件描述符 fd 上感兴趣的事件

short revents; // 文件描述符 fd 上当前实际发生的事件

} pollfd_t;

poll 改变了文件描述符集合的描述方式，使用了 pollfd 结构而不是 select 的 fd_set 结构，使得 poll 支持的文件描述符集合限制远大于 select 的 1024

【参数说明】

struct pollfd *fds fds 是一个 struct pollfd 类型的数组，用于存放需要检测其状 态的 socket 描述符，并且调用 poll 函数之后 fds 数组不会被清空；一个 pollfd 结构 体表示一个被监视的文件描述符，通过传递 fds 指示 poll() 监视多个文件描述符。其 中，结构体的 events 域是监视该文件描述符的事件掩码，由用户来设置这个域，结构 体的 revents 域是文件描述符的操作结果事件掩码，内核在调用返回时设置这个域

nfds_t nfds 记录数组 fds 中描述符的总数量

【返回值】

int 函数返回 fds 集合中就绪的读、写，或出错的描述符数量，返回 0 表示超时，返回 -1 表示出错；

Epoll

epoll 在 Linux2.6 内核正式提出，是基于事件驱动的 I/O 方式，相对于 select 来说， epoll 没有描述符个数限制，使用一个文件描述符管理多个描述符，将用户关心的文件 描述符的事件存放到内核的一个事件表中，这样在用户空间和内核空间的 copy 只需一 次。

Linux 中提供的 epoll 相关函数如下：

int epoll_create(int size);

int epoll_ctl(

int epfd,

```
int op, int fd,

struct epoll_event *event

);

int epoll_wait(

int epfd,

struct epoll_event * events,

int maxevents,

int timeout

);
```

1).epoll_create 函数创建一个 epoll 句柄，参数 size 表明内核要监听的描述符数量。 调用成功时返回一个 epoll 句柄描述符，失败时返回-1。

2).epoll_ctl 函数注册要监听的事件类型。四个参数解释如下：

epfd 表示 epoll 句柄

op 表示 fd 操作类型，有如下 3 种

EPOLL_CTL_ADD 注册新的 fd 到 epfd 中

EPOLL_CTL_MOD 修改已注册的 fd 的监听事件

EPOLL_CTL_DEL 从 epfd 中删除一个 fd

fd 是要监听的描述符

event 表示要监听的事件

epoll_event 结构体定义如下：

```
struct epoll_event {

__uint32_t events; /* Epoll events */
```

```
epoll_data_t data; /* User data variable */

};

typedef union epoll_data {

void *ptr;

int fd;

__uint32_t u32;

__uint64_t u64;

} epoll_data_t;
```

3). epoll_wait 函数等待事件的就绪，成功时返回就绪的事件数目，调用失败时返回 -1，等待超时返回 0。

⑴epfd 是 epoll 句柄

⑵events 表示从内核得到的就绪事件集合

⑶maxevents 告诉内核 events 的大小

⑷timeout 表示等待的超时事件

epoll 是 Linux 内核为处理大批量文件描述符而作了改进的 poll，是 Linux 下多路复用 IO 接口 select/poll 的增强版本，它能显著提高程序在大量并发连接中只有少量活跃 的情况下的系统 CPU 利用率。原因就是获取事件的时候，它无须遍历整个被侦听的描述 符集，只要遍历那些被内核 IO 事件异步唤醒而加入 Ready 队列的描述符集合就行了。

epoll 除了提供 select/poll 那种 IO 事件的水平触发（Level Triggered）外，还提供 了边缘触发（Edge Triggered），这就使得用户空间程序有可能缓存 IO 状态，减少 epoll_wait/epoll_pwait 的调用，提高应用程序效率。

⑴水平触发（LT）：默认工作模式，即当 epoll_wait 检测到某描述符事件就绪并 通知应用程序时，应用程序可以不立即处理该事件；下次调用 epoll_wait 时，会 再次通知此事件

⑵边缘触发（ET）： 当 epoll_wait 检测到某描述符事件就绪并通知应用程序时， 应用程序必须立即处理该事件。如果不处理，下次调用 epoll_wait 时，不会再次 通知此事件。（直到你做了某些操作导致该描述符变成未就绪状态了，也就是说边 缘触发只在状态由未就绪变为就绪时只通知一次）。

LT 和 ET 原本应该是用于脉冲信号的,可能用它来解释更加形象。Level 和 Edge 指的就 是触发点，Level 为只要处于水平，那么就一直触发，而 Edge 则为上升沿和下降沿的 时候触发。比如：0->1 就是 Edge，1->1 就是 Level。

ET 模式很大程度上减少了 epoll 事件的触发次数，因此效率比 LT 模式下高。

5. linux 内核调度详细说一下

1.1、调度策略

定义位于 linux/include/uapi/linux/sched.h 中

SCHED_NORMAL：普通的分时进程，使用的 fair_sched_class 调度类

SCHED_FIFO：先进先出的实时进程。当调用程序把 CPU 分配给进程的时候，它把该进程描述符保留在运行队列链表的当前位置。此调度策略的进程一旦使用 CPU 则一直运行。如果没有其他可运行的更高优先级实时进程，进程就继续使用 CPU，想用多久就用多久，即使还有其他具有相同优先级的实时进程处于可运行状态。使用的是 rt_sched_class 调度类。

SCHED_RR：时间片轮转的实时进程。当调度程序把 CPU 分配给进程的时候，它把该进程的描述符放在运行队列链表的末尾。这种策略保证对所有具有相同优先级的 SCHED_RR 实时进程进行公平分配 CPU 时间，使用的 rt_sched_class 调度类

SCHED_BATCH：是 SCHED_NORMAL 的分化版本。采用分时策略，根据动态优先级，分配 CPU 资源。在有实时进程的时候，实时进程优先调度。但针对吞吐量优化，除了不能抢占外与常规进程一样，允许任务运行更长时间，更好使用高速缓存，适合于成批处理的工作，使用的 fair_shed_class 调度类

SCHED_IDLE：优先级最低，在系统空闲时运行，使用的是 idle_sched_class 调度类，给 0 号进程使用

SCHED_DEADLINE：新支持的实时进程调度策略，针对突发型计算，

并且对延迟和完成时间敏感的任务使用，基于 EDF（earliest deadline first）,使用的是 dl_sched_class 调度类。

1.2、调度触发

调度的触发主要有两种方式，一种是本地定时中断触发调用 scheduler_tick 函数，然后使用当前运行进程的调度类中的 task_tick，另外一种则是主动调用 schedule，不管是哪一种最终都会调用到__schedule 函数，该函数调用 pick_netx_task，通过 rq->nr_running ==rq->cfs.h_nr_running 判断出如果当前运行队列中的进程都在 cfs 调度器中，则直接调用 cfs 的调度类（内核代码里面这一判断使用了 likely 说明大部分情况都是满足该条件的）。如果运行队列不都在 cfs 中，则通过优先级 stop_sched_class->dl_sched_class->rt_sched_class->fair_sched_class->idle_sched_class 遍历选出下一个需要运行的进程。然后进程任务切换。

处于 TASK_RUNNING 状态的进程才会被进程调度器选择，其他状态不会进入调度器。系统发生调度的时机如下：

à调用 cond_resched()时

à显式调用 schedule()时

à从中断上下文返回时

当内核开启抢占时，会多出几个调度时机如下：

à在系统调用或者中断上下文中调用 preemt_enable()时（多次调用系统只会在最后一次调用时会调度）

à在中断上下文中，从中断处理函数返回到可抢占的上下文时

2、CFS 调度

该部分代码位于 linux/kernel/sched/fair.c 中

定义了 const struct sched_classfair_sched_class，这个是 CFS 的调度类定义的对象。其中基本包含了 CFS 调度的所有实现。

CFS 实现三个调度策略：

1> SCHED_NORMAL 这个调度策略是被常规任务使用

2> SCHED_BATCH 这个策略不像常规的任务那样频繁的抢占，以牺牲交互性为代价下，因而允许任务运行更长的时间以更好的利用缓存，这种策略适合批处理

3> SCHED_IDLE 这是 nice 值甚至比 19 还弱，但是为了避免陷入优先级导致问题，这个问题将会死锁这个调度器，因而这不是一个真正空闲定时调度器

CFS 调度类：

n enqueue_task(…) 当任务进入 runnable 状态，这个回调将把这个任务的调度实体（entity）放入红黑树并且增加 nr_running 变量的值

n dequeue_task(…) 当任务不再是 runnable 状态，这个回调将会把这个任务的调度实体从红黑树中取出，并且减少 nr_running 变量的值

n yield_task(…) 除非 compat_yield sysctl 是打开的，这个

回调函数基本上就是一个 dequeue 后跟一个 enqueue,这那种情况下，他将任务的调度实体放入红黑树的最右端

n check_preempt_curr(…) 这个回调函数是检查一个任务进入 runnable 状态是否应该抢占当前运行的任务

n pick_next_task(…) 这个回调函数选出下一个最合适运行的任务

n set_curr_task(…) 当任务改变他的调度类或者改变他的任务组，将调用该回调函数

n task_tick(…) 这个回调函数大多数是被 time tick 调用。他可能引起进程切换。这就驱动了运行时抢占

2.1、CFS 调度

Tcik 中断，主要会更新调度信息，然后调整当前进程在红黑树中的位置。调整完成以后如果当前进程不再是最左边的叶子，就标记为 Need_resched 标志，中断返回时就会调用 scheduler()完成切换、否则当前进程继续占用 CPU。从这里可以看出 CFS 抛弃了传统时间片概念。Tick 中断只需要更新红黑树。

红黑树键值即为 vruntime，该值通过调用 update_curr 函数进行更新。这个值为 64 位的变量，会一直递增，__enqueue_entity 中会将 vruntime 作为键值将要入队的实体插入到红黑树中。__pick_first_entity 会将红黑树中最左侧即 vruntime 最小的实体取出。

6. 如何计算 struct 占用的内存?

1.每个成员按其类型大小和指定对齐参数 n 中较小的一个进行对齐

2.确定的对齐参数必须能够整除起始地址（或偏移量）

3.偏移地址和成员占用大小均需对齐

4.结构体成员的对齐参数为其所有成员使用的对齐参数的最大值

5.结构体总长度必须为所有对齐参数的整数倍

```
#include<stdio.h>
```

```
struct test

{

char a;

int b;

float c;

};


int main(void)

{

printf("char=%d\n",sizeof(char));

printf("int=%d\n",sizeof(int));
```

```
printf("float=%d\n",sizeof(float));

printf("struct test=%d\n",sizeof(struct test));

return 0;

}
```

执行结果为 1，4，4，12

占用内存空间的计算过程：

对齐参数为 4。假设结构体的起始地址为 0x0

a 的类型为 char，因此所占内存空间大小为 1 个字节，小于对齐参数 4，所以选择 1 为对齐数，而地址 0x0 能够被 1 整除，所以 0x0 为 a 的起始地址，占用空间大小为 1 个字节；

b 的类型为 int ，所占内存空间大小为 4 个字节，与对齐参数相同，因此 4 为对齐数，0x1 不能被 4 整除，因此不能作为 b 的起始地址，依次往下推，只能选用 0x4 作为 b 的起始地址，因此中间会空出 3 个字节的空余空间；

c 的类型为 float，占用 4 个字节，因此 4 为对齐数，0x8 能被 4 整除，所以 c 的起始地址为 0x8

因此整个结构体占用的内存大小为 12 字节。

7. mysql 为什么要使用 B+树作为索引呢？

b 树的特点:

一个 M 阶的 b 树具有如下几个特征： (如下图 M=3)(下文的关键字可以理解为 有效数据,而不是单纯的索引)

定义任意非叶子结点最多只有 M 个儿子，且 M>2；

根结点的儿子数为[2, M]；

除根结点以外的非叶子结点的儿子数为[M/2, M]，向上取整；(儿子数:[2,3])

非叶子结点的关键字个数=儿子数-1；(关键字=2)

所有叶子结点位于同一层；

k 个关键字把节点拆成 k+1 段，分别指向 k+1 个儿子，同时满足查找树的大小关系。(k=2)

有关 b 树的一些特性，注意与后面的 b+树区分：

关键字集合分布在整颗树中；

任何一个关键字出现且只出现在一个结点中；

搜索有可能在非叶子结点结束；

其搜索性能等价于在关键字全集内做一次二分查找；

b+树，是 b 树的一种变体，查询性能更好。m 阶的 b+树的特征：

有 n 棵子树的非叶子结点中含有 n 个关键字（b 树是 n-1 个），这些关键字不保存数据，只用来索引，所有数据都保存在叶子节点（b 树是每个关键字都保存数据）。

所有的叶子结点中包含了全部关键字的信息，及指向含这些关键字记录的指针，且叶子结点本身依关键字的大小自小而大顺序链接。

所有的非叶子结点可以看成是索引部分，结点中仅含其子树中的最大（或最小）关键字。

通常在 b+树上有两个头指针，一个指向根结点，一个指向关键字最小的叶子结点。

同一个数字会在不同节点中重复出现，根节点的最大元素就是b+树的最大元素。

选用 B+树作为数据库的索引结构的原因有

B+树的中间节点不保存数据，是纯索引,但是 B 树的中间节点是保存数据和索引的,相对来说,B+树磁盘页能容纳更多节点元素，更“矮胖”；

B+树查询必须查找到叶子节点，B 树只要匹配到即可不用管元素位置，因此 b+树查找更稳定（并不慢）；

对于范围查找来说，B+树只需遍历叶子节点链表即可，B 树却需要重复地中序遍历,在项目中范围查找又很是常见的

增删文件（节点）时，效率更高，因为 B+树的叶子节点包含所有关键字，并以有序的链表结构存储，这样可很好提高增删效率。

8. 请解释下为什么鹿晗发布恋情的时候，微博系统会崩溃，如何解决？

鹿晗首先是一个明星，流量明星。粉丝量众多，所以，他已公布恋情，瞬间的流量很大。但是我们要注意到，这里面有一个问题。就是这个瞬间流量增大，增的不仅是浏览量。如果仅仅是阅读，我们只需把鹿晗的这条微博放入 Redis 缓存，以微博技术，不可能挂得了的吧。

这个之所以微博挂掉，是因为这个时间段，转发 + 评论量非常的大，并不是只有阅读量大。

另外针对明星的微博，会有一个消息推送功能。第一时间热点数据，只要你联的有网，都能够收到推送。

最后总结如下:

1. 获取微博通过 pull 方式还是 push 方式

2. 发布微博的频率要远小于阅读微博

3. 流量明星的发微博，和普通博主要区分对待，比如在 sharding 的时候，也要考虑这个因素

9. 求任意一颗二叉树最长路径长度

样例:如下所示

图 1 树的最长路径长度为 4，图 2 的最长路径长度为 7，图 1 最长路径经过根节点，顶点为 1，图 2 不经过，顶点为 3

思路:

树中任意两个节点之间，连接起来的路径最长。方法就是求出每

个节点的左子树和右子树的高度，两者相加就是当前节点的最长路径，然后比较每个节点的最长路径，最大的就是结果

实现方法:

定义一个静态变量 MaxLength 记录每一步最大长度，采取前序遍历来遍历每一个节点，在遍历过程中，对当前节点的最长路径进行比较，对于每一个节点最长路径求法，先求出它左子树和右子树的高度(节点数最多的路径),然后相加即为当前节点最长路径

```
static Integer MaxLength=0;//记录最长路径

//遍历整棵树，得到最长路径

public void getLength(TreeNode t){

if(t!=null){

MaxLength=Math.max(LengthTree(t),MaxLength);

getLength(t.lchild);
```

```
getLength(t.rchild);

}

}

//得到当前节点的最长路径

public int LengthTree(TreeNode t){

if (t==null)

return 0;

int left=heighTree(t.lchild);

int right=heighTree(t.rchild);

int CurMax=left+right;

return CurMax;
```

```
}

//求二叉树最大高度

public int heighTree(TreeNode t){

if (t==null)

return 0;

else

return
Math.max(heighTree(t.lchild),heighTree(t.rchild))+1;

}
```

10. redis 中的网络 IO 有了解过吗，它是单线程的还是多线程的，为什么要用单线程。

redis 采用网络 IO 多路复用技术来保证在多连接的时候，系统的高吞吐量。

多路-指的是多个 socket 连接，复用-指的是复用一个线程。多路复用主要有三种技术：select，poll，epoll。epoll 是最新的也是目前最好的多路复用技术。

这里“多路”指的是多个网络连接，“复用”指的是复用同一个线程。采用多路 I/O

复用技术可以让单个线程高效的处理多个连接请求（尽量减少网络 IO 的时间消耗），且 Redis 在内存中操作数据的速度非常快（内存内的操作不会成为这里的性能瓶颈），主要以上两点造就了 Redis 具有很高的吞吐量。

因为 Redis 是基于内存的操作，CPU 不是 Redis 的瓶颈，Redis 的瓶颈最有可能是机器内存的大小或者网络带宽。既然单线程容易实现，而且 CPU 不会成为瓶颈，所以采用单线程。